Øvelse 1 Trunk-based development

I denne øvelse sætter I et GitHub-repo op så det understøtter trunk-based development med branch protection, og I praktiserer arbejdsflowet i par.

Forudsætninger

- GitHub-konto
- Git installeret + VS Code
- En makker (arbejdes i par)

Del 1 — Opret og konfigurér repo (10 min)

1. **Opret et nyt repo**
   Én fra parret opretter et nyt public repo på GitHub. Tilføj en `README.md` og en simpel `app.py` med en enkelt Flask-route.

2. **Aktivér branch protection på `main`**
   Gå til `Settings → Branches → Add rule`. Sæt `main` som branch name pattern og aktivér:
   ✓ Require a pull request before merging
   ✓ Require approvals: 1
   ✓ Require status checks to pass (hvis I har CI)
   ✓ Do not allow bypassing the above settings

3. **Tilføj makker som collaborator**
   Gå til `Settings → Collaborators` og invitér din makker.

Del 2 — Praktisér trunk-based workflow (20 min)

4. **Klon repo og lav en kortlivet branch**
   I trunk-based laves korte branches — max 1–2 dage. Navngiv efter opgave, ikke person.

```
git clone <repo-url>
git checkout -b # eks: add-health-endpoint
# lav en lille ændring i app.py
git add . && git commit -m "feat: tilføj /health endpoint"
git push origin add-health-endpoint
```

5. **Åbn en pull request og bed makker om review**
   Gå til GitHub og åbn en PR mod `main`. Makker reviewer og approver. Forsøg at pushe direkte til `main` — hvad sker der?

6. **Merge og slet branch straks**
   Trunk-based handler om hyppige, korte integrationer. Branch slettes efter merge — `main` er altid deploybar.

```
# Efter merge på GitHub:
git checkout main && git pull
git branch -d add-health-endpoint
```

7. **Byt roller og gentag**
   Den anden laver nu en lille ændring og åbner en PR. Gentag flowet mindst én gang hver.

> **Refleksion:** Hvad er forskellen på at pushe direkte til `main` og at bruge en kortlivet branch med PR? Hvornår giver trunk-based mening vs. GitHub Flow?